

LPII - Componentes Adicionais e Interface com Subclasses

1 – Ampliação da JanelaCadastroFilmes para suportar o Cadastro das Subclasses

Nesta seção, vamos ilustrar a representação de duas subclasses da entidade Filme na JanelaCadastroFilmes. Vamos especificar duas especializações (subclasses) da entidade [Filme](#):

- [FilmeCompanhiaCinematográfica](#)
- [FilmeProvedoraStreaming](#)

Vários filmes produzidos por uma companhia cinematográfica, como por exemplo a [20th Century Fox](#), são indicados todos os anos para concorrer à premiação do Oscar.

Na entidade [FilmeCompanhiaCinematográfica](#), são representados os atributos: oscar de melhor filme, e nome dos ganhadores de Oscar de melhor: diretor, ator principal e atriz principal. Para não estender demais, o número de atributos da entidade, foram excluídas as premiações para atores coadjuvantes e demais premiações do Oscar. O atributo de melhor filme tem preenchimento obrigatório, dado que um filme não ganhador, pode ser representado com o valor [false](#). Os atributos com os nomes de ganhadores de Oscar não tem preenchimento obrigatório, dado que para um dado filme tanto o diretor, quanto o ator e a atriz principal podem não ter sido premiados com um Oscar.

Os atributos da subclasse FilmeCompanhiaCinematográfica são os seguintes:

```
private boolean oscar_melhor_filme;
private String oscarMelhorDiretor;
private String oscarMelhorAtor;
private String oscarMelhorAtriz;
```

Provedoras de streaming, como por exemplo a [NetFlix](#), tem disponibilizado produções como filmes (com um único episódio) e ou como séries (com vários episódios). Na subclasse [FilmeProvedoraStreaming](#), são representados os seguintes enumerados e atributos:

```
public enum ProvedoraStreaming {NetFlix, AmazonPrimeVideo, HBOGo, GooglePlayStore};
public enum Produção {filme, série};

private ProvedoraStreaming provedora;
private Produção produção;
private int total_episódios;
```

No script sql, a ordem de remoção das tabelas deve priorizar as tabelas que dependem de outras tabelas e, portanto, as tabelas que representam as subclasses, que contém uma chave estrangeira para a chave da superclasse, devem ser removidas antes da tabela que representa a superclasse. A seguir, os seguintes comandos de remoção de tabelas existentes:

```
DROP TABLE IF EXISTS Atuações;
DROP TABLE IF EXISTS Amigos;
DROP TABLE IF EXISTS Atores;
DROP TABLE IF EXISTS FilmesCompanhiasCinematograficas;
DROP TABLE IF EXISTS FilmesProvedorasStreaming;
DROP TABLE IF EXISTS Filmes;
```

As tabelas que representarm as subclasses da entidade Filme, são incluídas no script sql, logo após a definição da tabela Filmes, pois possuem uma chave estrangeira associada à chave da tabela Filmes:

```
CREATE TABLE FilmesCompanhiasCinematográficas (
    OscarMelhorFilme BOOLEAN NOT NULL,
    OscarMelhorDiretor VARCHAR(30),
    OscarMelhorAtor VARCHAR(30),
    OscarMelhorAtriz VARCHAR(30),
    FilmeId INT NOT NULL,
    FOREIGN KEY (FilmeId) REFERENCES Filmes(Sequencial));

CREATE TABLE FilmesProvedorasStreaming (
    ProvedoraStreaming INT NOT NULL,
    Produção INT NOT NULL,
    TotalEpisódios INT NOT NULL,
    FilmeId INT NOT NULL,
    FOREIGN KEY (FilmeId) REFERENCES Filmes(Sequencial));
```

A seguir, é ilustrada a JanelaCadastroFilmes preenchida com um filme de companhia cinematográfica.

The screenshot shows a window titled "Cadastrar Filmes" with a tabbed interface. The "Filme de Companhia Cinematográfica" tab is selected. The form contains the following fields and values:

- Filmes Cadastrados:** A dropdown menu showing "[1] Titanic - Oscar Melhor Filme".
- Identificador Sequencial:** A text box containing the value "1".
- Título:** A text box containing the value "Titanic".
- Gênero:** A dropdown menu showing "romance".
- Ano:** A text box containing the value "1997".
- Atuações no Filme:** A text box containing the values "Leonardo DiCaprio [1974]" and "Kate Winslet [1975]".
- Oscar de Melhor Filme:** A checkbox that is checked.
- Oscar de Melhor Diretor:** A text box containing the value "James Cameron".
- Oscar de Melhor Ator:** An empty text box.
- Oscar de Melhor Atriz:** An empty text box.

At the bottom of the window, there are six buttons: "Inserir", "Consultar", "Alterar", "Remover", "Limpar", and "Atuações".

A seguir, é ilustrada a JanelaCadastroFilmes preenchida com um filme de provedora de streaming.

The screenshot shows a window titled "Cadastrar Filmes" with the following fields and controls:

- Filmes Cadastrados:** A dropdown menu showing "[2] Wandinha --- NetFlix - 8 episódios".
- Identificador Sequencial:** A text box containing the number "2".
- Título:** A text box containing "Wandinha".
- Gênero:** A dropdown menu showing "ficção".
- Ano:** A text box containing "2022".
- Atuações no Filme:** A list box containing "Jenna Ortega [2002]" and "Emma Myers [2002]".
- Tabbed Interface:** Two tabs are visible: "Filme de Companhia Cinematográfica" and "Filme de Provedora de Streaming". The second tab is active.
- Inside the active tab:**
 - Provedora de Streaming:** A dropdown menu showing "NetFlix".
 - Produção:** Radio buttons for "filme" and "série", with "série" selected.
 - Total de Episódios:** A text box containing "8".
- Buttons:** A row of buttons at the bottom: "Inserir", "Consultar", "Alterar", "Remover", "Limpar", and "Atores".

Na seção 2, é detalhada a configuração da [JanelaCadastroFilmes](#) para a visualização das abas contendo os atributos das subclasses, bem como, a criação de janelas auxiliares com painéis para para cada uma das subclasses.

Na seção 3, é explicado como as informações das subclasses passam a ser visualizadas no [filmes_cadastradosComboBox](#) da [JanelaCadastroFilmes](#).

Na seção 4, 5, 6 e 7 são abordadas as atualizações necessárias respectivamente para: a criação, alteração e remoção de filmes, e a limpeza do preenchimento do formulários de cadastro.

2 - Configurando a JanelaCadastroFilmes e as Janelas Auxiliares de Painéis para Subclasses

Na criação de um filme, são informados os atributos comuns da superclasse [Filme](#) e, adicionalmente, os atributos da subclasse para a qual se deseja criar o filme. Para informar os atributos específicos de uma das subclasses, é selecionada a aba que contém tais atributos.

Para realizar a seleção da aba que contém os atributos da subclasse desejada, bem como a visualização de seus atributos para preenchimento, é utilizada a seguinte estratégia:

- a janela auxiliar [PainelFilmeCompanhiaCinematográfica](#) é implementada, para representar os atributos da subclasse [FilmeCompanhiaCinematográfica](#);
- a janela auxiliar [PainelFilmeProvedoraStreaming](#) é implementada, para representar os atributos da subclasse [FilmeProvedoraStreaming](#);
- na [JanelaCadastroFilmes](#) é inserido o componente [especialização_filmeTabbedPane](#), ao qual são associadas as abas correspondentes a cada janela auxiliar de painel.

2.1 - A janela auxiliar PainelFilmeCompanhiaCinematográfica

A janela auxiliar [PainelFilmeCompanhiaCinematográfica](#) herda a classe [Panel](#) e, portanto, é criada a partir da opção [JPanel Form](#) do NetBeans. Nessa janela, são definidos seguintes os componentes: (a) um [CheckBox](#) para o atributo [oscar_filme](#); e (b) 3 [TextFields](#) para os atributos [oscar_diretor](#), [oscar_ator](#) e [oscar_atriz](#), bem como, seus respectivos [Labels](#).

O construtor desta janela auxiliar não tem parâmetros.

```
public PainelFilmeCompanhiaCinematográfica() {
    initComponents();
}
```

Adicionalmente, são definidos os métodos de leitura e alteração de cada um dos componentes da janela, bem como o método para limpeza dos campos.

```
public boolean isOscarMelhorFilme() { return oscar_filmeCheckBox.isSelected(); }

public void setOscarMelhorFilme(boolean oscar_melhor_filme) {
    oscar_filmeCheckBox.setSelected(oscar_melhor_filme);
}

public String getOscarMelhorDiretor() {
    String diretor = oscar_diretorTextField.getText();
    if (diretor.isEmpty()) return null;
    else return diretor;
}

public void setOscarMelhorDiretor(String diretor) { oscar_diretorTextField.setText(diretor); }

public String getOscarMelhorAtor() {
    String ator = oscar_atorTextField.getText();
    if (ator.isEmpty()) return null;
    else return ator;
}

public void setOscarMelhorAtor(String ator) { oscar_atorTextField.setText(ator); }

public String getOscarMelhorAtriz() {
    String atriz = oscar_atrizTextField.getText();
    if (atriz.isEmpty()) return null;
    else return atriz;
}

public void setOscarMelhorAtriz(String atriz) { oscar_atrizTextField.setText(atriz); }
```

2.2 - A janela auxiliar `PainelFilmeProvedoraStreaming`

A janela auxiliar `PainelFilmeProvedoraStreaming` é criada de forma equivalente, com os seguintes componentes: (a) um `ComboBox` para o atributo `provedora`; (b) um grupo de botões de radio (`RadioButton`) para o atributo `produção`; e (c) um `TextField` para o atributo `total_episódios`. E obviamente, os respectivos `Labels` para cada um desses componentes.

De forma similar, seu construtor não tem parâmetros. A implementação dos seus métodos de leitura e alteração dos componentes, bem como o método para limpeza dos campos, é ilustrada a seguir.

```
public ProvedoraStreaming getSelectedProvedoraStreaming() {
    Object provedora_streaming = provedoraComboBox.getSelectedItem();
    if (provedora_streaming != null) return (ProvedoraStreaming) provedora_streaming;
    else return null;
}

public void setSelectedProvedoraStreaming(ProvedoraStreaming provedora_streaming) {
    provedoraComboBox.setSelectedItem(provedora_streaming);
}

public Produção getSelectedProdução() {
    Produção produção = null;
    if (produçãoButtonGroup.getSelection() != null)
        produção = Produção.values()[produçãoButtonGroup.getSelection().getMnemonic()];
    return produção;
}

public void setSelectedProdução(int índice_produção) {
    switch(índice_produção) {
        case 0: filmeRadioButton.setSelected(true); break;
        case 1: sérieRadioButton.setSelected(true);
    }
}

public int getTotalEpisódios() {
    String total_episódios_str = total_episódiosTextField.getText();
    if (!total_episódios_str.isEmpty()) return Integer.parseInt(total_episódios_str);
    else return -1;
}

public void setTotalEpisódios(int total_episódios) {
    total_episódiosTextField.setText(total_episódios + "");
}

public void limparCampos() {
    provedoraComboBox.setSelectedIndex(-1);
    produçãoButtonGroup.clearSelection();
    total_episódiosTextField.setText("");
}
```

2.3 - O dados e o construtor da JanelaCadastroFilmes

Para adicionar as abas, criadas a partir dos painéis das janelas auxiliares ao componente [especialização_filmeTabbedPane](#) da [JanelaCadastroFilmes](#), são necessários novos dados na janela e ações adicionais no seu construtor.

```
ControladorCadastroFilmes controlador;
Filme[] filmes_cadastrados;
DefaultListModel modelo_atuações;
PainelFilmeCompanhiaCinematográfica filme_companhia_cinematográficaPainel;
PainelFilmeProvedoraStreaming filme_provedora_streamingPainel;

public JanelaCadastroFilmes(ControladorCadastroFilmes controlador) {
    this.controlador = controlador;
    filmes_cadastrados = Filme.getVisões();
    initComponents();
    modelo_atuações = (DefaultListModel)atuaçõesList.getModel();
    filme_companhia_cinematográficaPainel = new PainelFilmeCompanhiaCinematográfica();
    filme_provedora_streamingPainel = new PainelFilmeProvedoraStreaming();
    especialização_filmeTabbedPane.addTab("Filme de Companhia Cinematográfica",
        filme_companhia_cinematográficaPainel);
    especialização_filmeTabbedPane.addTab("Filme de Provedora de Streaming",
        filme_provedora_streamingPainel);
    limparCampos(null);
}
```

3 - Visualizando alguns Atributos das Subclasses no ComboBox com os Filmes Cadastrados

Observe que no [filmes_cadastradosComboBox](#) estão sendo informados alguns atributos das subclasses: (a) para filme de companhia cinematográfica, foi acrescentada a informação de oscar de melhor filme; e (b) para filme de provedora de streaming foram acrescentadas as informações do nome da provedora de streaming e do total de episódios. Obviamente a informação de oscar de melhor filme, só será mostrada se o filme conquistou essa categoria de oscar. O total de episódios é um campo obrigatório e, portanto, deverá ser informado com o valor 1, quando o filme de provedora de streaming não for uma série.

Para que essas informações, oriundas de atributos das subclasses apareçam no [filmes_cadastradosComboBox](#), os métodos construtor reduzido (utilizado para criar a visão), [getVisão](#) e [toString](#), herdados da superclasse, devem ser sobrepostos nas subclasses.

As implementações dos métodos sobrepostos, na subclasse [FilmeCompanhiaCinematográfica](#), são ilustradas a seguir.

```
public FilmeCompanhiaCinematográfica(int sequencial, String título,
    boolean oscar_melhor_filme) {
    super(sequencial, título);
    this.oscar_melhor_filme = oscar_melhor_filme;
}

public FilmeCompanhiaCinematográfica getVisão () {
    return new FilmeCompanhiaCinematográfica (sequencial, título, oscar_melhor_filme);
}

public String toString() {
    String str = "[" + sequencial + "]" + " " + título;
    if (oscar_melhor_filme) str += " - Oscar Melhor Filme";
    return str;
}
```

As implementações dos métodos sobrepostos, na subclasse [FilmeProvedoraStreaming](#), são ilustradas a seguir. O total de episódios só é mostrado no [filmes_cadastradosComboBox](#), quando seu valor é maior que 1, ou seja, somente para séries.

```
public FilmeProvedoraStreaming(int sequencial, String título, ProvedoraStreaming provedora,
    int total_episódios) {
    super(sequencial, título);
    this.provedora = provedora;
    this.produção = produção;
    this.total_episódios = total_episódios;
}

public FilmeProvedoraStreaming getVisão() {
    return new FilmeProvedoraStreaming(sequencial, título, provedora, total_episódios);
}

public String toString() {
    String str = "[" + sequencial + "]" + título + " --- " + provedora;
    if (total_episódios > 1) str += " - " + total_episódios + " episódios" ;
    return str;
}
```

4 - O Tratamento de Eventos para Inserir um Filme

No tratamento de eventos para inserir um filme, ocorrem as seguintes alterações nos seguintes métodos: (a) [obtemFilmeInformado](#) da classe [JanelaCadastroFilme](#); e (b) [buscarFilme](#) e [inserirFilme](#) da classe [Filme](#). As implementações de cada um desses métodos serão ilustradas em novas páginas, devido ao tamanho de seus códigos.

A implementação do método `obtemFilmeInformado` da classe `JanelaCadastroFilme`, é ilustrada a seguir.

```
private Filme obterFilmeInformado() {
    String sequencial_str = sequencialTextField.getText();
    int sequencial = 0; // se inserção de novo filme : sequencial ainda será atribuído pelo BD
    if (!sequencial_str.isEmpty()) sequencial = Integer.parseInt(sequencial_str);
    String titulo = tituloTextField.getText();
    if (titulo.isEmpty()) return null;
    Gênero gênero = null;
    if (gêneroComboBox.getSelectedItem() != null)
        gênero = (Gênero)gêneroComboBox.getSelectedItem();
    else return null;
    String ano_str = anoTextField.getText();
    int ano = -1;
    if (!ano_str.isEmpty()) ano = Integer.parseInt(ano_str);
    else return null;
    Filme filme = null;
    int índice_aba_seccionada = especialização_filmeTabbedPane.getSelectedIndex();
    switch (índice_aba_seccionada) {
        case 0:
            boolean oscarMelhorFilme =
                filme_companhia_cinematográficaPainel.isOscarMelhorFilme();
            String oscar_melhor_diretor =
                filme_companhia_cinematográficaPainel.getOscarMelhorDiretor();
            String oscar_melhor_ator =
                filme_companhia_cinematográficaPainel.getOscarMelhorAtor();
            String oscar_melhor_atriz =
                filme_companhia_cinematográficaPainel.getOscarMelhorAtriz();
            filme = new FilmeCompanhiaCinematográfica (sequencial, titulo, gênero, ano,
                oscarMelhorFilme, oscar_melhor_diretor, oscar_melhor_ator, oscar_melhor_atriz);
            break;
        case 1:
            ProvedoraStreaming provedora_streaming
                = filme_provedora_streamingPainel.getSelectedProvedoraStreaming();
            if (provedora_streaming == null) return null;
            Produção produção = filme_provedora_streamingPainel.getSelectedProdução();
            if (produção == null) return null;
            int total_episódios = filme_provedora_streamingPainel.getTotalEpisódios();
            if (total_episódios == -1) return null;
            filme = new FilmeProvedoraStreaming (sequencial, titulo, gênero, ano,
                provedora_streaming, produção, total_episódios);
    }
    return filme;
}
```

A implementação do método estático `buscarFilme` da classe `Filme`, é ilustrada a seguir.

```
public static Filme buscarFilme (int sequencial) {
    String sql = null;
    ResultSet lista_resultados = null;
    sql = "SELECT Título, Gênero, Ano FROM Filmes WHERE Sequencial = ?";
    String título = null;
    Gênero gênero = null;
    int ano = 0;
    boolean nacional = false;
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setInt(1, sequencial);
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) {
            título = lista_resultados.getString("Título");
            gênero = Gênero.values()[lista_resultados.getInt("Gênero")];
            ano = lista_resultados.getInt("Ano");
        }
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) { exceção_sql.printStackTrace(); }
    if (título == null) return null;
    sql = "SELECT OscarMelhorFilme, OscarMelhorDiretor, OscarMelhorAtriz, OscarMelhorAtriz"
        + " FROM FilmesCompanhiasCinematográficas WHERE FilmeId = ?";
    lista_resultados = null;
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setInt(1, sequencial);
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) {
            return new FilmeCompanhiaCinematográfica (sequencial, título, gênero, ano,
                lista_resultados.getBoolean("OscarMelhorFilme"),
                lista_resultados.getString("OscarMelhorDiretor"),
                lista_resultados.getString("OscarMelhorAtriz"),
                lista_resultados.getString("OscarMelhorAtriz"));
        }
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) { exceção_sql.printStackTrace(); }
    sql = "SELECT ProvedoraStreaming, Produção, TotalEpisódios FROM FilmesProvedorasStreaming"
        + " WHERE FilmeId = ?";
    lista_resultados = null;
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setInt(1, sequencial);
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) {
            return (new FilmeProvedoraStreaming (sequencial, título, gênero, ano,
                ProvedoraStreaming.values()[lista_resultados.getInt("ProvedoraStreaming")],
                Produção.values()[lista_resultados.getInt("Produção")],
                lista_resultados.getInt("TotalEpisódios")));
        }
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) { exceção_sql.printStackTrace(); }
    return null;
}
```

Uma observação importante é que será necessário retornar visões de objetos das subclasses no método `getVisões`. Para que isso seja possível será chamado o método acima `buscarFilme` no método `getVisões`, cuja implementação é mostrada na próxima página.

```

public static Filme[] getVisões () {
    String sql = "SELECT Sequencial FROM Filmes";
    ResultSet lista_resultados = null;
    ArrayList<Filme> visões = new ArrayList();
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) {
            visões.add(buscarFilme (lista_resultados.getInt("Sequencial")).getVisão());
        }
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) {exceção_sql.printStackTrace ();}
    return visões.toArray(new Filme[visões.size()]);
}

```

A implementação do método estático [inserirFilme](#) da classe [Filme](#), é ilustrada a seguir.

```

public static String inserirFilme (Filme filme) {
    String sql = "INSERT INTO Filmes (Título, Gênero, Ano) VALUES (?, ?, ?)";
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setString(1, filme.getTítulo());
        comando.setInt(2, filme.getGênero().ordinal());
        comando.setInt(3, filme.getAno());
        comando.executeUpdate();
        comando.close();
    } catch (SQLException exceção_sql) {
        exceção_sql.printStackTrace ();
        return "Erro na Inserção do Filme no BD";
    }
    int sequencial = últimoSequencial();
    if(filme instanceof FilmeCompanhiaCinematográfica) {
        FilmeCompanhiaCinematográfica filme_companhia_cinematográfica
            = (FilmeCompanhiaCinematográfica) filme;
        sql = "INSERT INTO FilmesCompanhiasCinematográficas"
            + " (OscarMelhorFilme, OscarMelhorDiretor,"
            + " OscarMelhorAtor, OscarMelhorAtriz, FilmeId) VALUES (?, ?, ?, ?, ?)";
        try {
            PreparedStatement comando = BD.conexão.prepareStatement(sql);
            comando.setBoolean(1, filme_companhia_cinematográfica.isOscarMelhorFilme());
            comando.setString(2, filme_companhia_cinematográfica.getOscarMelhorDiretor());
            comando.setString(3, filme_companhia_cinematográfica.getOscarMelhorAtor());
            comando.setString(4, filme_companhia_cinematográfica.getOscarMelhorAtriz());
            comando.setInt(5, sequencial);
            comando.executeUpdate();
            comando.close();
        } catch (SQLException exceção_sql) {
            exceção_sql.printStackTrace ();
            return "Erro na Inserção do FilmeCompanhiaCinematográfica no BD";
        }
    } else if(filme instanceof FilmeProvedoraStreaming) {
        FilmeProvedoraStreaming filme_provedora_streaming = (FilmeProvedoraStreaming) filme;
        sql = "INSERT INTO FilmesProvedorasStreaming"
            + " (ProvedoraStreaming, Produção, TotalEpisódios,"
            + " FilmeId) VALUES (?, ?, ?, ?)";
        try {
            PreparedStatement comando = BD.conexão.prepareStatement(sql);
            comando.setInt(1, filme_provedora_streaming.getProvedora().ordinal());
            comando.setInt(2, filme_provedora_streaming.getProdução().ordinal());
            comando.setInt(3, filme_provedora_streaming.getTotalEpisódios());
            comando.setInt(4, sequencial);
            comando.executeUpdate();
            comando.close();
        } catch (SQLException exceção_sql) {
            exceção_sql.printStackTrace ();
            return "Erro na Inserção do FilmeProvedoraStreaming no BD";
        }
    }
    return null;
}

```

5 - O Tratamento de Eventos para Consultar um Filme

No tratamento de eventos para consultar um filme, o único método alterado e ainda não ilustrado, é o método `consultarFilme` da classe `JanelaCadastroFilme`, cuja implementação é ilustrada a seguir.

```
private void consultarFilme(java.awt.event.ActionEvent evt) {
    Filme visão = (Filme) filmes_cadastradosComboBox.getSelectedItem();
    Filme filme = null;
    String mensagem_erro = null;
    int sequencial = -1;
    if (visão != null) {
        sequencial = visão.getSequencial();
        filme = Filme.buscarFilme(sequencial);
        if (filme == null) mensagem_erro = "Filme não cadastrado";
    } else mensagem_erro = "Nenhum filme selecionado";
    if (mensagem_erro == null) {
        sequencialTextField.setText(filme.getSequencial() + "");
        tituloTextField.setText (filme.getTitulo());
        generoComboBox.setSelectedItem(filme.getGênero());
        anoTextField.setText (filme.getAno() + "");
        atualizarListaAtuações(sequencial);
        if (filme instanceof FilmeCompanhiaCinematográfica) {
            especialização_filmeTabbedPane.setSelectedIndex(0);
            FilmeCompanhiaCinematográfica filme_companhia_cinematográfica
                = (FilmeCompanhiaCinematográfica) filme;
            filme_companhia_cinematográficaPainel.setOscarMelhorFilme
                (filme_companhia_cinematográfica.isOscarMelhorFilme());
            filme_companhia_cinematográficaPainel.setOscarMelhorDiretor
                (filme_companhia_cinematográfica.getOscarMelhorDiretor());
            filme_companhia_cinematográficaPainel.setOscarMelhorAtor
                (filme_companhia_cinematográfica.getOscarMelhorAtor());
            filme_companhia_cinematográficaPainel.setOscarMelhorAtriz
                (filme_companhia_cinematográfica.getOscarMelhorAtriz());
        } else if (filme instanceof FilmeProvedoraStreaming) {
            especialização_filmeTabbedPane.setSelectedIndex(1);
            FilmeProvedoraStreaming filme_provedora_streaming = (FilmeProvedoraStreaming) filme;
            filme_provedora_streamingPainel.setSelectedProvedoraStreaming
                (filme_provedora_streaming.getProvedora());
            filme_provedora_streamingPainel.setSelectedProdução
                (filme_provedora_streaming.getProdução().ordinal());
            filme_provedora_streamingPainel.setTotalEpisódios
                (filme_provedora_streaming.getTotalEpisódios());
        }
    } else informarErro (mensagem_erro);
}
```

6 - O Tratamento de Eventos para Alterar um Filme

No tratamento de eventos para alterar um filme, ocorrem as seguintes alterações nos seguintes métodos: (a) `alterarFilme` da classe `JanelaCadastroFilme`; e (b) `alterarFilme` da classe `Filme`.

Observe que como o usuário não consegue editar a chave `sequencial`, que é gerada pelo banco de dados, não é necessário consultar a visão alterada para modificar os seus atributos que não são chaves. Basta consultar a visão selecionada no ComboBox, porque a única forma que o usuário tem de informar a chave `sequencial` é consultar o objeto de interesse no ComboBox.

A implementação do método `alterarFilme` da classe `JanelaCadastroFilme` é ilustrada a seguir.

```
private void alterarFilme(java.awt.event.ActionEvent evt) {
    Filme filme = obtémFilmeInformado();
    String mensagem_erro = null;
    if (filme != null) mensagem_erro = controlador.alterarFilme(filme);
    else mensagem_erro = "Algum atributo do filme não foi informado";
    if (mensagem_erro == null) {
        Filme visão = (Filme) filmes_cadastradosComboBox.getSelectedItem();
        if (visão != null) {
            visão.setTitulo(filme.getTitulo());
            if (filme instanceof FilmeCompanhiaCinematográfica) {
                FilmeCompanhiaCinematográfica filme_companhia
                    = (FilmeCompanhiaCinematográfica) filme;
                FilmeCompanhiaCinematográfica visão_companhia
                    = (FilmeCompanhiaCinematográfica) visão;
                visão_companhia.setOscarMelhorFilme(filme_companhia.isOscarMelhorFilme());
            } else if (filme instanceof FilmeProvedoraStreaming) {
                FilmeProvedoraStreaming filme_streaming = (FilmeProvedoraStreaming) filme;
                FilmeProvedoraStreaming visão_streaming = (FilmeProvedoraStreaming) visão;
                visão_streaming.setProvedora(filme_streaming.getProvedora());
                visão_streaming.setTotalEpisódios(filme_streaming.getTotalEpisódios());
            }
            filmes_cadastradosComboBox.updateUI();
        }
    } else informarErro (mensagem_erro);
}
```

A implementação do método `alterarFilme` da classe `Filme` é ilustrada a seguir.

```
public static String alterarFilme (Filme filme) {
    String sql = "UPDATE Filmes SET Titulo = ?, Gênero = ?, Ano = ? WHERE Sequencial = ?";
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setString(1, filme.getTitulo());
        comando.setInt(2, filme.getGênero().ordinal());
        comando.setInt(3, filme.getAno());
        comando.setInt(4, filme.getSequencial());
        comando.executeUpdate();
        comando.close();
    } catch (SQLException exceção_sql) {
        exceção_sql.printStackTrace ();
        return "Erro na Alteração do Filme no BD";
    }
}

if(filme instanceof FilmeCompanhiaCinematográfica) {
    FilmeCompanhiaCinematográfica filme_companhia_cinematográfica
        = (FilmeCompanhiaCinematográfica) filme;
    sql = "UPDATE FilmesCompanhiasCinematográficas"
        + " SET OscarMelhorFilme = ?, OscarMelhorDiretor = ?, "
        + " OscarMelhorAtriz = ?, OscarMelhorAtriz = ? WHERE FilmeId = ?";
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setBoolean(1, filme_companhia_cinematográfica.isOscarMelhorFilme());
        comando.setString(2, filme_companhia_cinematográfica.getOscarMelhorDiretor());
        comando.setString(3, filme_companhia_cinematográfica.getOscarMelhorAtriz());
        comando.setString(4, filme_companhia_cinematográfica.getOscarMelhorAtriz());
        comando.setInt(5, filme_companhia_cinematográfica.getSequencial());
        comando.executeUpdate();
        comando.close();
    } catch (SQLException exceção_sql) {
        exceção_sql.printStackTrace ();
        return "Erro na Inserção do FilmeCompanhiaCinematográfica no BD";
    }
}

} else if(filme instanceof FilmeProvedoraStreaming) {
    FilmeProvedoraStreaming filme_provedora_streaming = (FilmeProvedoraStreaming) filme;
    sql = "UPDATE FilmesProvedorasStreaming SET ProvedoraStreaming = ?, Produção = ?, "
        + " TotalEpisódios = ? WHERE FilmeId = ?";
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setInt(1, filme_provedora_streaming.getProvedora().ordinal());
        comando.setInt(2, filme_provedora_streaming.getProdução().ordinal());
        comando.setInt(3, filme_provedora_streaming.getTotalEpisódios());
        comando.setInt(4, filme_provedora_streaming.getSequencial());
        comando.executeUpdate();
        comando.close();
    } catch (SQLException exceção_sql) {
        exceção_sql.printStackTrace ();
        return "Erro na Alteração do Filme no BD";
    }
}

return null;
}
```

7 - O Tratamento de Eventos para Remover um Filme

No tratamento de eventos para remover um filme, o único método alterado e ainda não ilustrado, é o método estático `removerFilme` da classe `Filme`, cuja implementação é ilustrada a seguir.

```
public static String removerFilme (Filme filme) {
    int sequencial = filme.getSequencial();
    Atuação.removerAtuações(sequencial);
    if (filme instanceof FilmeCompanhiaCinematográfica) {
        String sql = "DELETE FROM FilmesCompanhiasCinematográficas WHERE FilmeId = ?";
        try {
            PreparedStatement comando = BD.conexão.prepareStatement(sql);
            comando.setInt(1, sequencial);
            comando.executeUpdate();
            comando.close();
        } catch (SQLException exceção_sql) {
            exceção_sql.printStackTrace ();
            return "Erro na Remoção do FilmeOriginal do BD";
        }
    } else if (filme instanceof FilmeProvedoraStreaming) {
        String sql = "DELETE FROM FilmesProvedorasStreaming WHERE FilmeId = ?";
        try {
            PreparedStatement comando = BD.conexão.prepareStatement(sql);
            comando.setInt(1, sequencial);
            comando.executeUpdate();
            comando.close();
        } catch (SQLException exceção_sql) {
            exceção_sql.printStackTrace ();
            return "Erro na Remoção do FilmeOriginal do BD";
        }
    }
    String sql = "DELETE FROM Filmes WHERE Sequencial = ?";
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setInt(1, sequencial);
        comando.executeUpdate();
        comando.close();
    } catch (SQLException exceção_sql) {
        exceção_sql.printStackTrace ();
        return "Erro na Remoção do Filme do BD";
    }
    return null;
}
```

8 - O Tratamento de Eventos para Limpar Campos

No tratamento de eventos para limpar, a alteração ocorre no método auxiliar privado `limparCampos` da classe `JanelaCadastroFilme`. Observe que os métodos `limparCampos` dos painéis não recebem `null` como argumento porque não foram associados a evento gerado por clicar no botão `Limpar`, que não foi definido nesses painéis.

```
private void limparCampos(java.awt.event.ActionEvent evt) {
    sequencialTextField.setText("");
    tituloTextField.setText("");
    gêneroComboBox.setSelectedIndex(-1);
    anoTextField.setText("");
    modelo_atuações.clear();
    filme_companhia_cinematográficaPainel.limparCampos();
    filme_provedora_streamingPainel.limparCampos();
}
```